

DAY 1 – JAVA BASICS (IN-DEPTH + CODING HEAVY)

1. HOW JAVA PROGRAM EXECUTES (VERY IMPORTANT)

Think of Java like this flow:

None

Your Code → Compiler → Bytecode → JVM → Execution

Breakdown:

- You write `.java` file
- Compiler converts to `.class` (bytecode)
- JVM runs it → platform independent 

2. STRUCTURE OF JAVA PROGRAM

Java

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello Sameer");  
    }  
}
```

Deep Understanding:

Part	Meaning
public	accessible everywhere
class	blueprint
static	no object needed
void	returns nothing
main	entry point
String[] args	command line inputs



3. VARIABLES & MEMORY THINKING

Java

```
int a = 10;
```

👉 Memory view:

None

a → 10

Java

```
int b = a;
```

```
b = 20;
```

👉 Now:

None

a → 10

b → 20

✅ Primitive types store values directly.

◆ DATA TYPES (INTERVIEW IMPORTANT)

✓ Primitive:

- int
- double
- char
- boolean

✓ Non-Primitive:

- String
 - Array
-

🧠 4. OPERATORS (WITH INTERVIEW TRAPS)

🔴 Post vs Pre Increment

Java

```
int i = 5;
```

```
System.out.println(i++); // 5
```

```
System.out.println(++i); // 7
```

👉 Why?

- `i++` → use first, then increment
 - `++i` → increment first, then use
-

Assignment inside IF (BIG TRAP)

Java

```
int a = 10;
```

```
if(a = 5) // ❌ ERROR (not boolean)
```

✅ Java expects **true/false**, not value

5. CONDITIONALS (INTERVIEW STYLE)

Java

```
int age = 20;
```

```
if(age >= 18) {  
    System.out.println("Adult");  
} else {  
    System.out.println("Minor");  
}
```

Multi-condition

Java

```
if(age > 18 && age < 60)
```

Operators:

- **&&** AND
 - **||** OR
 - **!** NOT
-



6. LOOPS (DEEP + PATTERN READY)

✓ FOR LOOP

Java

```
for(int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

Flow:

None

start → check → run → update → repeat

✓ WHILE LOOP

Java

```
int i = 0;  
while(i < 5) {  
    System.out.println(i);  
    i++;  
}
```

✓ DO-WHILE (rare but asked)

Java

```
int i = 0;  
  
do {  
    System.out.println(i);  
    i++;  
}
```

```
} while(i < 5);
```

✓ Runs at least once

7. DRY RUN SKILL (MOST IMPORTANT)

Interviewer checks: 👉 Can you *think step-by-step*?

Example:

Java

```
int sum = 0;
for(int i = 1; i <= 3; i++) {
    sum += i;
}
```

Execution:

None

i=1 → sum=1

i=2 → sum=3

i=3 → sum=6

DAY 1 CODING (VERY IMPORTANT)

Do ALL — no skipping.

✓ LEVEL 1 (BASICS)

Q1: Print 1 to 10

- Using for loop
 - Using while loop
-

Q2: Even or Odd

Input:

Java

int n = 7;

Output:

None

Odd

Q3: Sum from 1 to N

None

Input: 5

Output: 15

Q4: Multiplication Table

None

Input: 3

Output:

3 x 1 = 3

...

3 x 10 = 30



LEVEL 2 (LOGIC BUILDING)

Q5: Reverse a Number

None

Input: 1234

Output: 4321

👉 Hint:

None

```
lastDigit = n % 10
```

```
n = n / 10
```

Q6: Count Digits

None

Input: 12345

Output: 5

Q7: Check Prime Number (VERY IMPORTANT)

None

Input: 7 → Prime

Input: 8 → Not Prime

👉 Hint: Loop from 2 to $n-1$

Q8: Factorial

None

Input: 5

Output: 120

LEVEL 3 (INTERVIEW FOUNDATION)

Q9: Fibonacci Series

None

Input: 5

Output: 0 1 1 2 3

Q10: Strong Number (Optional but good)

145 $\rightarrow 1! + 4! + 5! = 145$ 



THINK LIKE INTERVIEWER

For every question:

- Can you trace it?
 - Can you optimize it?
 - Can you explain logic clearly?
-



DAY 2 – ARRAYS (IN-DEPTH + INTERVIEW READY)



1. WHAT IS ARRAY (DEEP UNDERSTANDING)

Think of array as:

None

```
Index:  0   1   2   3   4
Value: [2 | 8 | 1 | 5 | 9]
```

- ✓ Stored in **continuous memory**
 - ✓ Fast access using index
 - ✗ Fixed size (important limitation)
-



2. MEMORY VISUALIZATION (IMPORTANT)

Java

```
int[] arr = {10, 20, 30};
```

Memory:

None

```
arr → [10 | 20 | 30]
```

0 1 2

👉 Access = $O(1)$ (very fast)

3. ARRAY OPERATIONS

✓ Traversal (MOST IMPORTANT)

Java

```
for(int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

OR

Java

```
for(int num : arr) {  
    System.out.println(num);  
}
```

✓ Insertion (conceptual)

👉 Arrays are fixed → insertion is costly

✓ Deletion

👉 Shift elements left

4. CORE LOGIC PATTERNS (VERY IMPORTANT)

Pattern 1: Finding something (max, min, etc.)

 Always use a variable

Java

```
int max = arr[0];
```

```
for(int i = 1; i < arr.length; i++) {  
    if(arr[i] > max) {  
        max = arr[i];  
    }  
}
```

Pattern 2: Counting

Java

```
int count = 0;
```

```
for(int num : arr) {  
    if(num % 2 == 0) {  
        count++;  
    }  
}
```

Pattern 3: Two Pointer (VERY IMPORTANT)

Used in:

- Reverse array
- Pair sum
- Many interview problems

Java

```
int left = 0;
```

```
int right = arr.length - 1;
```

```
while(left < right) {
```

```
// swap
```

```
int temp = arr[left];
```

```
arr[left] = arr[right];
```

```
arr[right] = temp;
```

```
left++;
```

```
right--;
```

```
}
```



5. TIME COMPLEXITY (INTRO)



Interviewer favorite

Operation	Time
Access	$O(1)$
Traversal	$O(n)$
Search	$O(n)$



6. COMMON MISTAKES

✗ `i <= arr.length` → crash

✓ `i < arr.length`

✗ `max = 0` (wrong if negative numbers)

✓ `max = arr[0]`

✗ Forget swap logic

✓ Use temp variable



DAY 2 CODING (CORE INTERVIEW SET)

Do these carefully — repeat if needed



LEVEL 1 (Foundation)

Q1: Find Maximum Element

👉 Must use loop

Q2: Find Minimum Element

Q3: Sum of Array

Q4: Count Even & Odd



LEVEL 2 (LOGIC BUILDING)

Q5: Reverse Array (MUST USE TWO POINTER 🔥)

None

Input: [1,2,3,4]

Output: [4,3,2,1]

Q6: Linear Search

Return index of target

Q7: Check if Array is Sorted

None

[1,2,3,4] → Yes

[1,5,3] → No



LEVEL 3 (INTERVIEW IMPORTANT)

Q8: Second Largest Element 🔥

None

[10, 20, 5, 8]

Output: 10

👉 Hint:

- Track max and second max

Q9: Move All Zeros to End 🔥

None

Input: [0,1,0,3,12]

Output: [1,3,12,0,0]

👉 Hint: Use pointer technique

Q10: Find Duplicate (Basic Version)

None

[1,2,3,2]

Output: 2

INTERVIEW INSIGHT

👉 80% array problems are based on:

- Traversal ✓
 - Two pointers ✓
 - Tracking variables ✓
-

BONUS (VERY IMPORTANT THINKING)

For each problem ask:

1. Can I solve in one loop?
2. Can I reduce space?
3. Can I avoid nested loops?

DAY 3 – STRINGS (DEEP + INTERVIEW READY)

1. WHAT IS STRING (VERY IMPORTANT DIFFERENCE)

Java

```
String s = "Sameer";
```

- ✓ String = sequence of characters
 - ✓ Internally behaves like **char array**
 - ✓ **Immutable** (VERY IMPORTANT 🔥)
-

IMMUTABLE CONCEPT

Java

```
String s = "Hello";
```

```
s = s + " World";
```

👉 Java creates NEW object:

None

"Hello" → "Hello World"

! Old object not changed

2. STRING MEMORY (INTERVIEW CONCEPT)

Java

```
String a = "Sameer";
```

```
String b = "Sameer";
```

✓ Both point to same memory (String Pool)

Java

```
String c = new String("Sameer");
```

✗ Created in heap (different object)

3. IMPORTANT METHODS

✓ **Length**

Java

```
s.length();
```

✓ Char Access

Java

```
s.charAt(i);
```

✓ Compare (VERY IMPORTANT)

Java

```
a.equals(b); // ✓ correct
```

```
a == b; // ✗ compare reference
```

✓ Convert to Char Array

Java

```
char[] arr = s.toCharArray();
```

🧠 4. STRING TRAVERSAL (CORE OF ALL PROBLEMS)

Java

```
for(int i = 0; i < s.length(); i++) {  
  
    System.out.println(s.charAt(i));  
  
}
```

5. STRING BUILDER (OPTIMIZATION)

👉 Use when modifying string frequently

Java

```
StringBuilder sb = new StringBuilder("Hello");  
  
sb.append(" World");
```

✅ Faster

✅ Mutable

6. CORE PROBLEM PATTERNS

Pattern 1: Character Counting

Java

```
int count = 0;  
  
for(int i = 0; i < s.length(); i++) {  
  
    if(s.charAt(i) == 'a') {  
  
        count++;  
  
    }  
  
}
```

Pattern 2: Reverse String

Java

```
String rev = "";
```

```
for(int i = s.length() - 1; i >= 0; i--) {
```

```
    rev += s.charAt(i);
```

```
}
```

✓ Better version → use `StringBuilder`

Pattern 3: Palindrome (VERY IMPORTANT 🔥)

Java

```
String s = "madam";
```

```
int left = 0;
```

```
int right = s.length() - 1;
```

```
while(left < right) {
```

```
    if(s.charAt(left) != s.charAt(right)) {
```

```
        return false;
```

```
    }
```

```
    left++;
```

```
    right--;
```

```
}
```

```
return true;
```

DAY 3 CODING (INTERVIEW LEVEL BUILDING)

LEVEL 1 (Foundation)

Q1: Count Vowels

None

Input: "sameer"

Output: 4

Q2: Count Characters (frequency of a character)

Q3: Reverse String

Q4: Check Palindrome



LEVEL 2 (LOGIC BUILDING)

Q5: Remove Spaces

None

"hello world" → "helloworld"

Q6: Toggle Case

None

"HeLLo" → "hE11o"

Q7: Find First Non-Repeating Character

None

"aabbcd" → c



LEVEL 3 (IMPORTANT FOR INTERVIEWS)

Q8: Check Anagram 🔥🔥

None

`listen & silent → true`

👉 Hint:

- Convert to char array
 - Sort and compare
-

Q9: Longest Word in Sentence

Q10: String Compression

None

`aaabb → a3b2`

INTERVIEW THINKING (VERY IMPORTANT)

For every string problem ask:

- ✓ Can I use two pointers?
 - ✓ Can I use frequency array?
 - ✓ Can I optimize using StringBuilder?
-

COMMON MISTAKES

✗ Using `==` for strings

✓ Use `.equals()`

✗ String concatenation in loop (slow)

✓ Use `StringBuilder`

✗ Ignoring case sensitivity

✓ Handle properly

BONUS (VERY IMPORTANT)

ASCII Trick

Java

```
char c = 'A';
```

```
System.out.println((int)c); // 65
```

👉 Useful for:

- Case conversion
- Frequency array



DAY 4 – PATTERNS + NESTED LOOPS



1. CORE IDEA (VERY IMPORTANT)

👉 Pattern = Loops inside loops

None

Outer loop → Rows

Inner loop → Columns



VISUAL THINKING

Example:

None

👉 Think:

Row	Column
4	4



2. BASIC STRUCTURE

Java

```
for(int i = 1; i <= rows; i++) {
```

```
for(int j = 1; j <= cols; j++) {  
    System.out.print("*");  
}  
System.out.println();  
}
```

3. TYPES OF PATTERNS

1. Square
 2. Triangle
 3. Pyramid
 4. Number patterns
 5. Character patterns
-

DAY 4 CODING (STEP-BY-STEP)

LEVEL 1 (FOUNDATION)

Q1: Square Pattern

None

```
****  
****  
****  
****
```

👉 Logic:

- Outer = rows

- Inner = stars
-

Q2: Rectangle Pattern

None

```
*****
*****
*****
```

Q3: Right Triangle

None

```
*
**
***
****
```

👉 Inner loop depends on `i`

Q4: Reverse Triangle

None

```
****
***
**
*
```



LEVEL 2 (IMPORTANT LOGIC)

Q5: Number Triangle

None

1

12

123

1234

Q6: Same Number Pattern

None

1

22

333

4444

Q7: Reverse Number Triangle

None

1234

123

12

1



LEVEL 3 (INTERVIEW FAVORITE 🔥)

Q8: Pyramid Pattern

None

*

👉 Key concept:

- Spaces + stars

Q9: Inverted Pyramid

None

*

Q10: Diamond Pattern 🔥

None

*

*

HOW TO THINK (IMPORTANT)

For every pattern:

Step 1:

Count rows

Step 2:

Find pattern in:

- Spaces
- Stars / numbers

Step 3:

Link to loop variables



MOST IMPORTANT TRICK

👉 For Pyramid:

None

```
Spaces = n - i
```

```
Stars = 2*i - 1
```



COMMON MISTAKES

- ✗ Confusing rows with columns
 - ✗ Wrong inner loop condition
 - ✗ Forgetting `println()`
-



INTERVIEW CONNECTION

👉 Why patterns matter:

- Builds **loop control**
 - Helps in **matrix problems**
 - Improves **thinking clarity**
-



BONUS (VERY IMPORTANT)

Try this:

Q11: Floyd's Triangle

None

1

2 3

4 5 6

7 8 9 10

👉 Requires:

- Variable tracking across loops



DAY 5 – SEARCHING + SORTING (DEEP + INTERVIEW READY)



PART 1: SEARCHING



1. LINEAR SEARCH (REVISION BUT IMPORTANT)

👉 Search element by scanning whole array

Java

```
int linearSearch(int[] arr, int target) {  
    for(int i = 0; i < arr.length; i++) {
```



```
if(arr[i] == target) {  
    return i;  
}  
return -1;  
}
```

✓ Time: $O(n)$

🔴 2. BINARY SEARCH 🔥🔥 (TOP INTERVIEW QUESTION)

👉 Works only on **SORTED ARRAY**

🧠 IDEA

None

[1 2 3 4 5 6 7]

Target = 5

Step 1: middle = 4

Step 2: $5 > 4 \rightarrow$ go RIGHT

Step 3: found ✓

✓ CODE (VERY IMPORTANT)

Java

```
int binarySearch(int[] arr, int target) {  
    int left = 0;  
    int right = arr.length - 1;  
  
    while(left <= right) {
```

```
int mid = left + (right - left) / 2;
```

```
if(arr[mid] == target) {  
    return mid;  
}  
else if(arr[mid] < target) {  
    left = mid + 1;  
}  
else {  
    right = mid - 1;  
}  
}  
return -1;  
}
```

INTERVIEW TRAPS

- ✗ Using on unsorted array
- ✗ Infinite loop (`left < right` mistake)
- ✗ Wrong mid calculation

✓ Always use:

Java

```
int mid = left + (right - left) / 2;
```

TIME COMPLEXITY

Method	Time
Linear	$O(n)$
Binary	$O(\log n)$ 🔥



PART 2: SORTING



1. BUBBLE SORT (BASIC)

👉 Repeatedly swap adjacent elements



CODE

Java

```
void bubbleSort(int[] arr) {  
    int n = arr.length;  
  
    for(int i = 0; i < n - 1; i++) {  
        for(int j = 0; j < n - i - 1; j++) {  
            if(arr[j] > arr[j+1]) {  
                int temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
            }  
        }  
    }  
}
```



IDEA

None

After each loop → largest element goes to end

2. SELECTION SORT

👉 Pick minimum and swap

Java

```
void selectionSort(int[] arr) {  
    int n = arr.length;
```

```
    for(int i = 0; i < n - 1; i++) {  
        int minIndex = i;
```

```
        for(int j = i + 1; j < n; j++) {  
            if(arr[j] < arr[minIndex]) {  
                minIndex = j;  
            }  
        }  
    }
```

```
    int temp = arr[i];  
    arr[i] = arr[minIndex];  
    arr[minIndex] = temp;  
}  
}
```

3. INSERTION SORT

👉 Insert element at correct position

Java

```
void insertionSort(int[] arr) {  
    int n = arr.length;
```

```
    for(int i = 1; i < n; i++) {  
        int key = arr[i];  
        int j = i - 1;
```

```
while(j >= 0 && arr[j] > key) {  
    arr[j+1] = arr[j];  
    j--;  
}  
  
arr[j+1] = key;  
}  
}
```

SORTING COMPLEXITY

Algorithm	Time
Bubble	$O(n^2)$
Selection	$O(n^2)$
Insertion	$O(n^2)$

INTERVIEW INSIGHT

👉 Don't just code — understand:

- When to use Binary Search
 - Why sorting is needed
 - Optimization mindset
-

DAY 5 CODING TASKS



LEVEL 1

Q1: Implement Linear Search

Q2: Implement Binary Search 🔥

Q3: Check if array is sorted



LEVEL 2

Q4: Bubble Sort

Q5: Selection Sort

Q6: Insertion Sort



LEVEL 3 (INTERVIEW FAVORITES)

Q7: First Occurrence using Binary Search 🔥

None

`[1,2,2,2,3] → target = 2 → output = index 1`

Q8: Last Occurrence

Q9: Count occurrences using Binary Search

Q10: Search in Rotated Sorted Array 🔥🔥 (advanced)

None

[4, 5, 6, 7, 0, 1, 2]



CORE LEARNING TODAY

✓ Binary Search = MUST MASTER

✓ Sorting = builds logic

✓ Time Complexity = interview key



DAY 6 – RECURSION (DEEP + INTERVIEW READY)



1. WHAT IS RECURSION?

👉 A function calling itself

SIMPLE EXAMPLE

Java

```
void print(int n) {  
    if(n == 0) return; // base case
```

```
    System.out.println(n);  
    print(n - 1); // recursive call  
}
```

THINK LIKE THIS (VERY IMPORTANT)

Recursion =

None

Function → breaks into smaller problem → until base case

2. TWO MOST IMPORTANT PARTS

1. Base Case (STOP condition)

Java

```
if(n == 0) return;
```

! Without this → infinite loop → crash

2. Recursive Call

Java


```
print(n - 1);
```

👉 Moves towards base case

3. CALL STACK (VERY IMPORTANT)

Example:

Java

```
print(3);
```

Stack:

```
None  
print(3)  
print(2)  
print(1)  
print(0) → stop
```

Then it returns:

```
None  
1  
2  
3
```

4. TYPES OF RECURSION

1. PRINT BEFORE CALL

Java

```
System.out.println(n);  
print(n - 1);
```

Output:

None

3 2 1

2. PRINT AFTER CALL

Java

```
print(n - 1);  
System.out.println(n);
```

Output:

None

1 2 3

5. VERY IMPORTANT PATTERN

👉 90% recursion problems follow:

None

1. Base case
 2. Do something
 3. Recursive call
-



DAY 6 CODING (VERY IMPORTANT)



LEVEL 1 (FOUNDATION)

Q1: Print numbers 1 to N

Q2: Print numbers N to 1

Q3: Factorial using recursion

Java

$n! = n * (n-1)!$

Q4: Sum of N numbers

Q5: Power (x^n)

None

$2^3 = 8$



LEVEL 2 (CORE RECURSION THINKING)

Q6: Reverse an Array using recursion

👉 Hint:

- Use start + end index
-

Q7: Check Palindrome using recursion

Q8: Fibonacci using recursion

Java

$$f(n) = f(n-1) + f(n-2)$$

⚠ Important: This is slow (learn, but optimize later)

✅ LEVEL 3 (INTERVIEW IMPORTANT 🔥)

Q9: Binary Search using recursion

Q10: Sum of digits

None

$$123 \rightarrow 1+2+3 = 6$$

🧠 **MOST IMPORTANT SKILL (GAME CHANGER 🔥)**

DRY RUN RECURSION

Example:

Java

factorial(3)

Execution:

None

```
3 * factorial(2)
2 * factorial(1)
1 * factorial(0)
```

Return:

None

```
1 → 1 → 2 → 6
```

COMMON MISTAKES

- ✗ No base case
 - ✗ Wrong base condition
 - ✗ Infinite recursion
 - ✗ Not understanding stack
-

INTERVIEW INSIGHT

👉 Why recursion matters:

- Base of **Trees**
 - Base of **Backtracking** (important)
 - Base of **Dynamic Programming**
-

PRO TIP

Before writing code, always ask:

- ✓ What is smallest problem? (base case)
- ✓ How to reduce problem size?

DAY 8 – STACK + QUEUE (DEEP + INTERVIEW READY)

PART 1: STACK (LIFO)

1. WHAT IS STACK?

 **Last In First Out (LIFO)**

None

Push → Add

Pop → Remove

Peek → Top element

Visual:

None

Top ↓

[30]

[20]

[10]

👉 Remove → 30 first

2. BASIC OPERATIONS

✅ Using Java Stack (built-in)

Java

```
import java.util.Stack;
```

```
Stack<Integer> stack = new Stack<>();
```

```
stack.push(10);
```

```
stack.push(20);
```

```
stack.push(30);
```

```
System.out.println(stack.pop()); // 30
```

```
System.out.println(stack.peek()); // 20
```

✅ Using Array (Manual - Interview Favorite 🔥)

Java

```
class Stack {
```

```
int top = -1;
```

```
int[] arr = new int[100];
```

```
void push(int x) {  
    arr[++top] = x;  
}
```

```
int pop() {  
    return arr[top--];  
}
```

```
int peek() {  
    return arr[top];  
}  
}
```

3. STACK REAL USE CASE

👉 Expression handling:

- () balancing
 - Reverse problems
 - Function call (recursion internally uses stack)
-

STACK CODING PROBLEMS

✅ Q1: Reverse String using Stack

✅ Q2: Valid Parentheses   (VERY IMPORTANT)

None

Input: "()[]{}" → true

Input: "()" → false

👉 Use stack

✓ Q3: Next Greater Element (important)

🧠 PART 2: QUEUE (FIFO 🔥)

🔴 1. WHAT IS QUEUE?

👉 First In First Out (FIFO)

None

Add → rear

Remove → front

Visual:

None

Front → [10][20][30] ← Rear

Remove → 10 first



2. IMPLEMENTATION

✓ Using Java Queue

Java

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

```
Queue<Integer> q = new LinkedList<>();
```

```
q.add(10);
```

```
q.add(20);
```

```
q.add(30);
```

```
System.out.println(q.poll()); // 10
```

```
System.out.println(q.peek()); // 20
```

✓ Using Array (Concept)

Java

```
int[] q = new int[100];
```

```
int front = 0, rear = -1;
```

```
void add(int x) {
```

```
q[++rear] = x;
```

```
}
```

```
int remove() {
```

```
return q[front++];
```

```
}
```

3. QUEUE USE CASE

👉 Used in:

- BFS (Graphs, Trees)
 - Scheduling (OS)
 - Real-world queues
-

QUEUE CODING PROBLEMS

✅ Q4: Implement Queue

✅ Q5: Reverse Queue

✅ Q6: First Non-Repeating Character in Stream 

4. STACK VS QUEUE (IMPORTANT)

Feature	Stack	Queue
Order	LIFO	FIFO
Insert	Top	Rear
Delete	Top	Front



5. IMPORTANT INTERVIEW PATTERNS



Using STACK:

- Parenthesis problems ✓
 - Expression evaluation ✓
 - Monotonic stack ✓
-



Using QUEUE:

- BFS ✓
 - Sliding window ✓
 - Level-order traversal ✓
-



COMMON MISTAKES

- ✗ Forgetting empty check
 - ✗ Stack overflow (array)
 - ✗ Queue index confusion
-



DAY 8 TASK



MUST DO

1. Stack using array
2. Valid Parentheses 🔥

3. Queue using array
-



IMPORTANT

1. Reverse string using stack
 2. Reverse queue
-



INTERVIEW LEVEL

1. Next Greater Element
2. First non-repeating character



DAY 9 – HASHING (SUPER IMPORTANT



)



1. WHAT IS HASHING?



Storing data in a way that allows **FAST** access ($O(1)$)



SIMPLE IDEA

Instead of searching again & again:

❌ Brute Force:

None

for every element → check all others ($O(n^2)$)

✅ Hashing:

None

Store → Lookup instantly ($O(n)$)

2. HOW HASHING WORKS

👉 Value → Hash function → Index → Store

Think like:

None

Value: 5 → goes to index 5

Value: 10 → goes to index 10

3. HASHING IN JAVA

✅ 1. HashMap (MOST IMPORTANT)

Java

```
import java.util.*;
```

```
HashMap<Integer, Integer> map = new HashMap<>();
```

```
map.put(1, 10);
```

```
map.put(2, 20);
```

```
System.out.println(map.get(1)); // 10
```

✓ Common Methods

Java

```
map.put(key, value);  
map.get(key);  
map.containsKey(key);  
map.remove(key);
```

✓ 2. HashSet (for unique elements)

Java

```
HashSet<Integer> set = new HashSet<>();
```

```
set.add(1);  
set.add(1);  
set.add(2);
```

```
System.out.println(set); // [1,2]
```

✓ 3. Frequency Counting (MOST IMPORTANT 🔥)

Java

```
HashMap<Integer, Integer> map = new HashMap<>();
```

```
for(int num : arr) {  
    map.put(num, map.getOrDefault(num, 0) + 1);  
}
```



4. CORE INTERVIEW PATTERNS



Pattern 1: Frequency Map



Count occurrences



Pattern 2: Check existence



Seen before or not?



Pattern 3: Pair problems



Store complements



DAY 9 CODING (INTERVIEW FOCUSED)



LEVEL 1 (FOUNDATION)

Q1: Count frequency of elements

None

[1, 2, 2, 3, 3, 3]

Output:

1 → 1

2 → 2

3 → 3

Q2: Count characters in string

Q3: Find duplicate element

LEVEL 2 (IMPORTANT)

Q4: First Non-Repeating Element 🔥

None

[4, 5, 1, 2, 0, 4]

Output: 5

Q5: Intersection of Two Arrays

Q6: Check if array has duplicates

LEVEL 3 (INTERVIEW FAVORITE 🔥 🔥)

Q7: Two Sum (VERY IMPORTANT)

None

Input: [2,7,11,15], target = 9

Output: [0,1]

Optimized Code

Java

```
HashMap<Integer, Integer> map = new HashMap<>();
```

```
for(int i = 0; i < arr.length; i++) {  
    int complement = target - arr[i];
```

```
    if(map.containsKey(complement)) {  
        System.out.println(map.get(complement) + " " + i);  
    }
```

```
    map.put(arr[i], i);  
}
```

Q8: Subarray with given sum

Q9: Longest Consecutive Sequence

None

[100,4,200,1,3,2] → output: 4 (1,2,3,4)

Q10: Majority Element (> n/2)



TIME COMPLEXITY MAGIC

Problem	Before	After Hashing
Two Sum	$O(n^2)$	$O(n)$ ✓
Frequency	$O(n^2)$	$O(n)$ ✓



COMMON MISTAKES

- ✗ Forgetting `getOrDefault()`
 - ✗ Using map when not needed
 - ✗ Confusing key/value
-



WHEN TO USE HASHING?

Ask yourself:

- ✓ Need fast lookup?
- ✓ Checking duplicates?
- ✓ Counting frequency?

👉 Use Hashing 🔥



VERY IMPORTANT INTERVIEW TIP

👉 When interviewer asks:

"Can you optimize this?"

💡 Answer = **Use HashMap / HashSet**

🚀 DAY 10 – TREES (DEEP + INTERVIEW READY)

🧠 1. WHAT IS A TREE?

👉 Tree = Hierarchical data structure

Visual:

None



🔴 KEY TERMS (VERY IMPORTANT)

Term	Meaning
Root	Top node (10)
Parent	Node with children
Child	Node below parent

Leaf	Node with no children
Height	Longest path
Depth	Distance from root

2. BINARY TREE

👉 Each node has **max 2 children**

None

left child ← node → right child

✅ NODE STRUCTURE (VERY IMPORTANT)

Java

```
class Node {  
    int data;  
    Node left, right;  
  
    Node(int data) {  
        this.data = data;  
        left = right = null;  
    }  
}
```

3. TREE TRAVERSALS (MOST IMPORTANT)

These are asked in almost every interview!

1. INORDER (LNR)

None

Left → Node → Right

Java

```
void inorder(Node root) {  
    if(root == null) return;  
  
    inorder(root.left);  
    System.out.print(root.data + " ");  
    inorder(root.right);  
}
```

2. PREORDER (NLR)

None

Node → Left → Right

Java

```
void preorder(Node root) {  
    if(root == null) return;  
  
    System.out.print(root.data + " ");  
    preorder(root.left);  
    preorder(root.right);  
}
```

3. POSTORDER (LRN)

None

Left → Right → Node

Java

```
void postorder(Node root) {  
    if(root == null) return;  
  
    postorder(root.left);  
    postorder(root.right);  
    System.out.print(root.data + " ");  
}
```



4. LEVEL ORDER (BFS)

👉 Uses Queue

None

Level by level

Java

```
import java.util.*;  
  
void levelOrder(Node root) {  
    Queue<Node> q = new LinkedList<>();  
    q.add(root);  
  
    while(!q.isEmpty()) {  
        Node curr = q.poll();  
        System.out.print(curr.data + " ");  
  
        if(curr.left != null) q.add(curr.left);  
        if(curr.right != null) q.add(curr.right);  
    }  
}
```

5. IMPORTANT CONCEPT

👉 Trees = **Recursion + DFS + BFS**

- DFS → Inorder / Preorder / Postorder
 - BFS → Level order
-

6. BINARY SEARCH TREE (BST)

👉 Special tree with rule:

None

Left < Root < Right

Example:

None

```
      10
     /  \
    5    20
```

SEARCH IN BST

Java

```
boolean search(Node root, int key) {
    if(root == null) return false;

    if(root.data == key) return true;
    else if(key < root.data) return search(root.left, key);
    else return search(root.right, key);
}
```


✓ Time: $O(\log n)$

7. COMMON INTERVIEW PATTERNS

Pattern 1: Traversal

👉 Basic but must know 100%

Pattern 2: Height of Tree

Java

```
int height(Node root) {  
    if(root == null) return 0;  
  
    return 1 + Math.max(height(root.left), height(root.right));  
}
```

Pattern 3: Count Nodes

Java

```
int count(Node root) {  
    if(root == null) return 0;  
  
    return 1 + count(root.left) + count(root.right);  
}
```

Pattern 4: Sum of Nodes

DAY 10 CODING TASKS

LEVEL 1 (FOUNDATION)

Q1: Create a Binary Tree manually

Q2: Inorder traversal

Q3: Preorder traversal

Q4: Postorder traversal

LEVEL 2 (IMPORTANT)

Q5: Level Order traversal 

Q6: Height of Binary Tree

Q7: Count nodes

Q8: Sum of nodes

LEVEL 3 (INTERVIEW FAVORITES)

Q9: Search in BST

Q10: Check if tree is balanced

Q11: Diameter of Tree  (important)

INTERVIEW INSIGHT

👉 Most interview questions come from:

-  Traversals
 -  Recursion
 -  BST properties
-

COMMON MISTAKES

-  Forget base case ($\text{root} == \text{null}$)
-  Wrong traversal order
-  Not visualizing tree properly

PRO THINKING (GAME CHANGER)

When solving tree problems:

Ask: ☒ Can I solve using recursion?

☒ What is left + right result?

☒ How to combine results?

DAY 11 – HEAP + PRIORITY QUEUE

1. WHAT IS HEAP?

👉 Heap = **Complete Binary Tree** with special property

TYPES OF HEAP

☒ 1. Min Heap

👉 Smallest element at top

None



10 15

✓ 2. Max Heap

👉 Largest element at top

None

```
  20
 /  \
10   15
```

🧠 2. IMPORTANT PROPERTY

👉 It is a **COMPLETE TREE**

✓ Filled from left to right

✓ No gaps allowed

🧠 3. HOW HEAP IS STORED?

👉 Internally stored in **Array**

Formula:

None

Parent = i

Left = $2*i + 1$

Right = $2*i + 2$



4. PRIORITY QUEUE (JAVA SHORTCUT



👉 Java gives heap directly



MIN HEAP (default)

Java

```
import java.util.*;
```

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

```
pq.add(10);
```

```
pq.add(5);
```

```
pq.add(20);
```

```
System.out.println(pq.poll()); // 5 (smallest)
```



MAX HEAP

Java

```
PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
```



5. IMPORTANT OPERATIONS

Operation	Time
Insert	$O(\log n)$

Delete	$O(\log n)$
Peek	$O(1)$ 

6. CORE INTERVIEW PATTERNS

Pattern 1: TOP K ELEMENTS

 Most common interview question

Pattern 2: Kth Largest / Smallest

Pattern 3: Merge K sorted arrays

Pattern 4: Sliding window problems

DAY 11 CODING TASKS

LEVEL 1 (FOUNDATION)

Q1: Insert elements into Min Heap

Q2: Print elements in sorted order using heap

Q3: Convert array to heap

LEVEL 2 (IMPORTANT)

Q4: Kth Smallest Element 🔥

None

`[7,10,4,3,20,15], k = 3 → output: 7`

👉 Use Max Heap of size k

Q5: Kth Largest Element 🔥

Q6: Sort a nearly sorted array

LEVEL 3 (INTERVIEW FAVORITES 🔥 🔥)

Q7: Top K Frequent Elements 🔥

None

$[1, 1, 1, 2, 2, 3]$, $k=2 \rightarrow [1, 2]$

Q8: Merge K Sorted Arrays

Q9: Find Median from Data Stream 🔥🔥

👉 Use 2 heaps:

- Max heap (left)
 - Min heap (right)
-

Q10: Sliding Window Maximum 🔥🔥



CORE INTERVIEW IDEA

👉 Heap is used when:

- ✓ Need smallest/largest quickly
 - ✓ Need top K elements
 - ✓ Need dynamic sorting
-



VERY IMPORTANT THINKING

Instead of sorting full array:

- ✗ Sort $\rightarrow O(n \log n)$
- ✓ Heap $\rightarrow O(n \log k)$ (OPTIMAL 🔥)

COMMON MISTAKES

- ✗ Using full sorting instead of heap
 - ✗ Confusing min heap vs max heap
 - ✗ Not limiting heap size (for k problems)
-

INTERVIEW STRATEGY

When you see:

- 👉 “Top K”
- 👉 “Kth largest/smallest”
- 👉 “Frequent elements”
- 💡 Think → **HEAP** 🔥

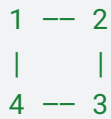
DAY 12 – GRAPHS (DEEP + INTERVIEW READY)

1. WHAT IS A GRAPH?

- 👉 Graph = collection of **nodes (vertices)** + **edges**
-

VISUAL

None



👉 Nodes = 1,2,3,4

👉 Edges = connections

2. TYPES OF GRAPHS

Directed Graph

None



👉 Direction matters

Undirected Graph

None



👉 Both ways

Weighted Graph

None

1 --(5)-- 2

👉 Edge has cost

3. GRAPH REPRESENTATION (VERY IMPORTANT)

1. ADJACENCY MATRIX

Java

```
int[][] graph = new int[n][n];
```

👉 Uses 2D array

2. ADJACENCY LIST (MOST USED)

Java

```
ArrayList<ArrayList<Integer>> adj = new ArrayList<>();
```

```
for(int i = 0; i < n; i++) {  
    adj.add(new ArrayList<>());  
}
```

```
// Add edge  
adj.get(0).add(1);  
adj.get(1).add(0);
```

- ✓ Efficient
 - ✓ Used in interviews
-

4. GRAPH TRAVERSAL (CORE 🔥 🔥)

1. DFS (Depth First Search)

👉 Go deep first (uses recursion / stack)

✓ CODE

Java

```
void dfs(int node, boolean[] visited, ArrayList<ArrayList<Integer>> adj) {  
    visited[node] = true;  
    System.out.print(node + " ");  
  
    for(int neighbor : adj.get(node)) {  
        if(!visited[neighbor]) {  
            dfs(neighbor, visited, adj);  
        }  
    }  
}
```

2. BFS (Breadth First Search)

👉 Level by level (uses queue)

CODE

Java

```
import java.util.*;

void bfs(int start, ArrayList<ArrayList<Integer>> adj) {
    boolean[] visited = new boolean[adj.size()];
    Queue<Integer> q = new LinkedList<>();

    q.add(start);
    visited[start] = true;

    while(!q.isEmpty()) {
        int node = q.poll();
        System.out.print(node + " ");

        for(int neighbor : adj.get(node)) {
            if(!visited[neighbor]) {
                visited[neighbor] = true;
                q.add(neighbor);
            }
        }
    }
}
```

5. CORE INTERVIEW PATTERNS

Pattern 1: Traversal (DFS/BFS)

Pattern 2: Connected Components

👉 Count number of groups

Pattern 3: Cycle Detection

Pattern 4: Shortest Path

(BFS for unweighted graph)

6. IMPORTANT PROBLEMS

Q1: Number of Islands (VERY FAMOUS)

👉 Matrix + DFS

Q2: Detect Cycle in Graph

Q3: Shortest Path (BFS)

Q4: Topological Sort (DAG)

Q5: Graph Coloring

7. IMPORTANT CONCEPTS

VISITED ARRAY (VERY IMPORTANT)

 Prevent infinite loops

Java

```
boolean[] visited = new boolean[n];
```

QUEUE vs STACK

Algorithm	Uses
BFS	Queue
DFS	Stack / Recursion

DAY 12 CODING TASKS

LEVEL 1 (FOUNDATION)

Q1: Create graph using adjacency list

Q2: BFS traversal

Q3: DFS traversal

 LEVEL 2 (IMPORTANT)

Q4: Count connected components

Q5: Detect cycle in undirected graph

Q6: Shortest path (BFS)

 LEVEL 3 (INTERVIEW FAVORITES)

Q7: Number of Islands 

Q8: Topological Sort 

Q9: Course Schedule problem  

Q10: Dijkstra Algorithm (advanced)



INTERVIEW THINKING

👉 When you see:

- Grid problem → DFS/BFS
 - Shortest path → BFS / Dijkstra
 - Dependency → Topological sort
-



COMMON MISTAKES

- ✗ Not using visited array
 - ✗ Infinite loop
 - ✗ Graph representation confusion
-



PRO TIP (VERY IMPORTANT)

👉 80% graph problems =

- ✓ BFS
- ✓ DFS

Master these → you solve MOST problems 🔥

DAY 13 – DYNAMIC PROGRAMMING (DP)



1. WHAT IS DP?

👉 DP = Optimization using storing results



SIMPLE IDEA

Instead of solving same problem again & again:

✗ Recursion (slow)

None

$f(5) \rightarrow f(4) + f(3)$

$f(4) \rightarrow f(3) + f(2)$

👉 $f(3)$ computed multiple times ✗

✓ DP (store result)

None

$f(3)$ calculated once ✓

reuse it later ✓



2. WHEN TO USE DP?

👉 Ask yourself:

✓ Same subproblem repeating?

✓ Need optimal solution?

➡ Use DP 🔥



3. TWO WAYS TO DO DP



1. MEMOIZATION (Top-Down)

👉 Recursion + Cache

Java

```
int fib(int n, int[] dp) {  
    if(n <= 1) return n;  
  
    if(dp[n] != -1) return dp[n];  
  
    dp[n] = fib(n-1, dp) + fib(n-2, dp);  
    return dp[n];  
}
```



2. TABULATION (Bottom-Up)

👉 Build from base

Java

```
int fib(int n) {  
    int[] dp = new int[n+1];  
  
    dp[0] = 0;  
    dp[1] = 1;  
  
    for(int i = 2; i <= n; i++) {  
        dp[i] = dp[i-1] + dp[i-2];  
    }  
  
    return dp[n];  
}
```

4. MOST IMPORTANT PATTERN (REMEMBER THIS)

👉 DP steps:

1. Define dp[]
2. Find recurrence relation
3. Base case
4. Fill DP

5. CLASSIC EXAMPLE

Fibonacci

👉 Brute: $O(2^n)$ ❌

👉 DP: $O(n)$ ✅

6. CORE DP PROBLEM TYPES

TYPE 1: 1D DP

👉 Linear problems

- Fibonacci

- Climbing stairs ✓
 - House robber ✓
-

TYPE 2: 2D DP

👉 Grid problems

- Unique paths ✓
 - Minimum path sum ✓
-

TYPE 3: SUBSEQUENCE DP

👉 Very important

- Longest common subsequence 🔥
 - LIS 🔥
-

DAY 13 CODING TASKS

LEVEL 1 (FOUNDATION)

Q1: Fibonacci (Memoization + Tabulation)

Q2: Climbing Stairs 🔥

None

$n = 3 \rightarrow \text{ways} = 3$

👉 Jump 1 or 2 steps

Q3: Frog Jump (basic)

✅ LEVEL 2 (IMPORTANT)

Q4: House Robber 🔥

None

[2, 7, 9, 3, 1]

Max stolen = 12

👉 Cannot take adjacent

Q5: Minimum steps to reach 1

Q6: Maximum sum subarray (Kadane's Algorithm 🔥)

✅ LEVEL 3 (INTERVIEW FAVORITES 🔥 🔥)

Q7: Longest Common Subsequence 🔥

Q8: Longest Increasing Subsequence 🔥

Q9: 0/1 Knapsack 🔥🔥

Q10: Coin Change 🔥🔥

DP THINKING (GAME CHANGER 🔥)

HOW TO APPROACH DP

Ask:

- 👉 What is the decision?
 - 👉 What is changing (index, value, capacity)?
 - 👉 Can I break into smaller problems?
-

Example:

Climbing Stairs:

None

$\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$

TIME COMPLEXITY MAGIC

Approach	Time
Recursion	$O(2^n)$ ❌

DP	$O(n)$ ✓
----	----------



COMMON MISTAKES

- ✗ Not identifying recurrence
 - ✗ Wrong base case
 - ✗ Confusing memo vs tabulation
-



INTERVIEW STRATEGY

When you see:

- 👉 Minimum / Maximum
 - 👉 Count ways
 - 👉 Optimization
- 💡 Think → DP 🔥



DAY 14 – INTERVIEW PROBLEM SOLVING



HOW THIS PHASE WORKS

From now on:

✓ You solve **real interview-level questions**

✓ I guide you on:

- Approach
- Optimization
- Pattern recognition

✓ Focus shifts to:

- Problem solving ✓
 - Speed ✓
 - Clarity ✓
-



MOST IMPORTANT SKILL NOW

👉 Every problem:

1. Understand problem
 2. Think brute force
 3. Optimize
 4. Code clean
 5. Explain properly
-



TODAY'S SET (REAL INTERVIEW QUESTIONS)



Q1: Two Sum (MUST MASTER)

None

Input: [2,7,11,15], target = 9

Output: [0,1]

👉 You already saw this — now solve **clean + fast**

✅ Q2: Best Time to Buy & Sell Stock 🔥

None

[7, 1, 5, 3, 6, 4]

Output: 5

👉 Buy low, sell high

👉 Hint:

- Track minimum price
-

✅ Q3: Kadane's Algorithm 🔥 🔥

None

[-2, 1, -3, 4, -1, 2, 1, -5, 4]

Output: 6 (subarray: [4, -1, 2, 1])

👉 Max subarray sum

✅ Q4: Move Zeroes 🔥

None

[0, 1, 0, 3, 12] → [1, 3, 12, 0, 0]

👉 Two pointer problem



Q5: Valid Parentheses 🔥 🔥

None

```
"()[{}]" → true
```

```
"[]" → false
```

👉 Stack



Q6: Reverse Linked List 🔥 🔥

👉 MOST ASKED



Q7: Binary Tree Level Order Traversal



👉 BFS + Queue



Q8: Kth Largest Element 🔥

👉 Heap



Q9: Climbing Stairs 🔥

👉 DP problem



Q10: Number of Islands 🔥🔥

👉 Graph + DFS/BFS



INTERVIEW PATTERN SUMMARY

Pattern	Example
Hashing	Two Sum
Sliding Window	Stock, Subarray
Stack	Parentheses
Two Pointer	Move Zeros
Heap	Kth Largest
Tree/BFS	Level order
Graph	Islands
DP	Climbing stairs



HOW YOU SHOULD SOLVE

For EACH question:

- 👉 Step 1: Write brute force
- 👉 Step 2: Optimize
- 👉 Step 3: Think pattern
- 👉 Step 4: Code



WHAT YOU MUST AVOID

- ✗ Jumping directly to code
 - ✗ Not thinking edge cases
 - ✗ Not knowing time complexity
-



YOUR TASK

Solve these gradually (not all at once)